



扫码
查看更

 **系列导航：** 上一篇 [08.架构思维] 帮你产出了一份「能让 AI 看得懂的架构文档」。这一篇带你完成开干前最后一步——**技术思维：架构定了，用什么工具实现。**

兄弟们，今天聊一件可能颠覆你认知的事。

很多人觉得，做 AI 产品最难的那关是「技术」——选什么语言、用什么框架、怎么部署……一想起来就头大，干脆放弃。

说句实话，恰好反了。

06 讲为什么做、07 讲做什么、08 讲怎么走——这三篇全是「想清楚」的活，真烧脑。今天 09 这一篇「用什么搭」，反而是整个实战篇里最不烧脑的一篇。

不信？往下读你就知道为什么。

看完这一篇，你会拿到：

- 一份「非程序员 AI 时代选型」的完整方法（四原则 + 三个变化）
- 一份判断 GitHub 开源项目靠不靠谱的速查表（star / 更新 / issue 三个信号）
- 一份照搬就能用的 fuxi 真实技术栈（含每个内容渠道的具体抓取工具）
- 一套「不上向量也能找到关联」的轻量级办法（fuxi 同款，零运维）
- 一份你今天就能让 AI 出的「架构 + 选型」初稿提示词

01 技术思维 ≠ 写代码

先把这件事讲清楚，免得你又把它跟「学编程」混了。

很多人一听「技术思维」就脑补成「我要学 Python、学 React、学数据库……」——其实不是。

技术思维管的是一件特别小的事：

架构定下来「东西怎么走」之后，下一步是「每一步用什么工具走」——技术思维就是教你怎么挑这些工具。

注意，是挑工具，不是造工具。这两件事的难度差了几个数量级。

维度	造工具（传统程序员）	挑工具（你）
难度	高，要懂底层原理	低，会比较就行
时间	数月 to 数年	几小时
谁干	全世界顶级程序员	你 + AI

你的角色	工程师	采购员
------	-----	-----

AI 时代，普通人做产品要练的不是「造工具」，是「当好一个采购员」。

 **一句话总结：**技术思维 = 学会当一个会挑工具的采购员，不是去学怎么造工具。

02 那段「爬网页」的代码，是 AI 现场写的吗

先用一个具体例子，把「挑工具」这件事确实。

还记得 06 讲的 fuxi 吗？你扔一个链接给它，它把网页抓下来、整理成笔记。

那段「抓下来」的代码，是 AI 现场从零写的吗？

不是。

抓网页这件事，本质要做的是：发请求、拿 HTML、解析里面的标题正文、处理重定向、绕反爬……这套东西，全世界程序员已经写过、调过、维护了几十年，统称爬虫库。

AI 干的事是：从现成的爬虫库里挑一个、调用它，然后帮你写几行胶水代码，把它和 fuxi 拼起来。

「胶水代码」是程序员行话——说白了就是专门把别人写好的东西粘到一起的几行小代码，本身没啥逻辑，作用类似装家具时的螺丝：单看没用，但少了它整个柜子就散架。这种活，恰好是 AI 写起来又快又准的活。

AI 没有、也不会、也不需要从零给你写一个爬虫库。

往大了说，你做任何一个产品，80% 的代码都不是 AI 现场原创——而是调用别人已经写好的工具。AI 干的是装修工人的活，不是发明家的活：

- 1 帮你挑工具
- 2 帮你组装工具（写胶水代码把各种库串起来）
- 3 帮你调试工具（报错时 Google + 改配置）

那这些「现成工具」，从哪挑？——GitHub。

 **一句话总结：**你想装个画框，不会先去炼铁造一把锤子——你去五金店挑一把。写软件也一样。

03 GitHub 是什么，以及怎么判断一个项目靠不靠谱

GitHub 这个词你可能听过无数次，但没真用过。这一节给你彻底讲明白。

GitHub 是什么

简单说，GitHub 是全世界最大的开源代码仓库。「开源」= 作者把代码免费公开，任何人都能拿走用、改、二次发布，甚至商用，当然这里还跟开源协议挂钩。

几个数字感受下：

- 截至 2026 年，4 亿+ 个仓库，几乎涵盖你能想到的所有功能
- 你听过的所有大公司——Google、Meta、字节、阿里——都把核心工具放在上面
- 全世界几乎所有程序员都在上面有账号

你可以把 GitHub 想成三个东西的合体：

-  图书馆：每个项目都是一本书，写着代码怎么写

- 五金店：每个项目都是一件工具，照着说明书拿来用
- 菜市场：明码标价（其实免费）、有评分（star 数）、有评论（issue）

为啥免费？程序员世界有个共识——好工具应该共享。靠这种共享，整个软件行业才能像复利一样越积越多。所以做新项目时，能用现成开源工具的地方，默认就用现成的。

怎么判断一个开源项目靠不靠谱

同一件事可能有几十个开源项目都能做——比如「爬网页」我随便搜搜就能列出十几个库。怎么挑？

看三个信号就够了：

信号	看什么	判断标准
★ Star 数	项目首页右上角的星星数	1k+ 起步，1w+ 算成熟的头部
📅 最近更新	最后一次 commit 时间	半年内有更新算活的，2 年没动的基本没跟上 AI 时代了
💬 Issue 活跃度	有人提问、有人回复	维护者最近一个月有回复算健康

三项都好的项目放心用。三项都不行的，再有名也别碰——可能哪天就没人管了，你想改都找不到人问。

我自己挑库的私心三步法：先按 star 排前 3、再剔除掉一年没更新的、最后让 AI 把剩下的对比一下使用门槛。三步选完不到 10 分钟，AI 时代不用纠结太多。

 **一句话总结：**挑开源项目就三个信号——star 多 + 更新近 + 维护者活，三好学生直接用。

04 AI 时代选型逻辑，变了三件事

工具有了，挑的逻辑也得跟着变。

传统程序员选技术栈，有自己一套思路；但作为一个借 AI 做产品的普通人，你的选型逻辑跟他们完全不一样。

维度	程序员视角	你的视角（AI 时代）
选语言看什么	我会的、性能好的、酷的	AI 训练数据里最多的
选架构看什么	可扩展性、并发性能	跑得起来、改得动
选库看什么	最新的、最潮的	AI 最熟的、文档最全的
容忍度	完美主义，死磕细节	能跑就行，出问题能改就行

我做 fuxi 的时候，第一版差点用 Rust。

当时刷推特看到一堆人吹 Rust 多快多安全，我就鬼迷心窍想试试。结果第一天就被卡住：依赖装不上、报错看不懂、问 Claude 给我的代码每次都不一样、Google 半天找不到对应的中文资料……

第二天我老老实实换成 Python，当天晚上就把 fuxi 第一版跑起来了。

那次踩坑让我总结出—条铁律：AI 时代不要装，优先选 AI 最熟的栈。

💡 **一句话总结：**选型不是给程序员看的，是给 AI + 你这对组合看的——AI 熟 + 最好你能改，缺一个都不行。

05 非程序员选型四原则

把上面的逻辑落到具体动作，就是这四条。但在讲具体原则之前，先把这一切的总纲亮出来：

做产品，优先走轻量方案。

能不上的组件就别上、能用文件就别上数据库、能本地跑就别上云、能用现成工具就别造轮子——这是我自己做了好几个产品总结出的最高原则，也是 fuxi 从抓取到存储到索引全栈「极简到不能再极简」背后的同一句话。下面四条原则，都是它的具体展开。

原则一：选 AI 训练数据里最多的语言

优先级：Python > TypeScript / JavaScript > 其他。

为啥？这两门语言的代码在 AI 训练语料里数量级最大，AI 写起来又快又准。冷门语言（Rust / Elixir / Haskell）AI 也会，但老出错、文档少、社区小，踩坑成本极高。

原则二：选纯本地能跑的方案

能不依赖云、不开服务器，就别开。

理由很简单：服务器要钱、要运维、要域名、要备案、要部署、要监控……每一件事都是一个新坑。MVP 阶段你的目标是最快验证想法，不是练运维。

fuxi 整个跑在我自己的 Mac 上，零服务器、零部署、零运维。

原则三：选零运维的存储

存数据的优先级：

文件 > 数据库

是的，你没看错。

很多人一想到「存数据」就上数据库——MySQL、Postgres、MongoDB……根本不需要。80% 的个人产品，纯文件就够了。fuxi 整个知识库就是一堆 Markdown 文件，零数据库，十几万字内容跑得飞快。

非要用数据库时，选零运维的：SQLite > Postgres > MySQL。SQLite 就是一个文件，不用启服务、不用建用户、不用管端口。

原则四：选社区大的工具

判断标准就是上一节的三个信号：star 多 + 更新近 + 维护者活。

社区大有两个超大的好处：

- 1 AI 训练数据里这个工具的相关代码多，AI 写起来准
- 2 遇到问题 Google + AI 都能答上，你自己也能解决问题

冷门工具反过来——出问题没人答、AI 也含糊其辞，你就卡死了。



💡 一句话总结：四个字——主流、本地、简单、大众。

06 fuxi 的选型清单（照搬就能用）

光讲原则没感觉，看 fuxi 真实的选型清单——按「抓取 / 存储 / 索引 + 编排」三层拆开，每个工具都给到具体名字，你今天就能上 GitHub 搜出来。

抓取层：按渠道挑，没有银弹

我做 fuxi 之前以为「抓网页」是一件统一的事。做了才发现，每个内容渠道有自己专属的最佳工具，没有一个库能通吃。

渠道	用的工具	怎么挑的
普通网页 / 博客	Claude Code 内置 WebFetch	直接调用 Claude Code 自带，零额外依赖
微信公众号	Jina Reader	公众号反爬严，自己写正则会被锤；Jina 提供托管 reader API，抓下来直接是干净 Markdown
GitHub	gh CLI	GitHub 官方命令行，登录后能直接读 README、issue、release，不用折腾 API
YouTube	yt-dlp	视频抓取库里 star 最多的（80k+），抓字幕、抓元信息都靠它
X / Twitter	opencli twitter 系列	X 反爬最狠，用社区现成工具，自己造轮子等于送死

这张表的关键洞察：抓取这件事不要想着「写一个通用爬虫框架打天下」——选每个渠道社区最成熟的专用工具，让 AI 帮你把它们串起来。

存储层：纯文件 + 分层文件夹，零数据库

项目	选了什么	为啥这么选
主数据	纯 Markdown 文件	人能肉眼读、git 能 diff、AI 能直接喂——三个角色都满足
目录分层	raw/（原文）+ notes/（原子笔记）+ wiki/（编译成品）+ briefs/（选题）	按「生熟程度」分层，不按时间分层——AI 一眼就能定位
数据库	❌ 没用	个人知识库十几万字，文件 + Markdown frontmatter（写在文件最上面、用 --- 框起来的一小段标签信息）完全够

向量库 / embedding	❌ 没用	个人知识库完全不需要
--------------------	------	------------

fxi 整个跑下来连 SQLite 都没碰。简单就是最大的稳定。

索引 + 编排层：脚本干机械活，AI 干语义活

项目	选了什么	为啥这么选
自动索引	一个生成索引的小脚本（AI 写的 Python）	扫所有笔记的 frontmatter，按分类 + 标签自动生成 INDEX.md——机械活让脚本干
关联推荐	一个关联打分脚本（AI 写的 Python）	「标签重叠 + 标题关键词」打分推荐，双向写入 <code>related</code> 字段（下一节细讲）
检索	一个关键词搜索脚本 + Claude 长上下文	关键词够准；剩下的语义判断扔给 Claude 在长上下文里做
AI 模型	Claude（命令行调用）	我日常最顺手、写中文笔记最准的模型
工作流编排	Claude 语义工作流（写在 <code>CLAUDE.md</code> ）	不写复杂状态机，让 AI 直接读 <code>CLAUDE.md</code> 里的规则执行
运行环境	本地 Mac	零服务器、零部署、零运维

为啥脚本只干机械活？ 因为机械事（扫文件、统计标签、生成索引）脚本快又稳；语义事（判断这条笔记和那条有没有关联、要不要打这个标签）扔给 AI 才准。**机械归脚本、语义归 Claude**——这是我用了几个月总结出的最重要一条分工原则。

看到没？fxi **零数据库、零向量库、零服务器、零部署**——一个 **「极简到不能再极简」** 的栈。

很多人会怀疑：这能行吗？

我用了好几个月，**比我之前折腾过的任何复杂方案都顺手**。原因就一条：**简单就是最大的稳定**。每多一个组件，就多一个出问题的点；每多一个云服务，就多一项支出。

你做自己第一个产品，记住一句话：**先把烂版本跑起来，再去想优化**。

💡 **一句话总结：***fxi 的栈一句话概括——专用抓取工具（按渠道挑） + 纯 Markdown 文件 + 几个 AI 写的 Python 小脚本（索引 / 关联 / 搜索） + Claude。*

07 不上向量也能找到关联：fxi 是怎么做的

这一节单独拎出来讲——因为这是 fxi 跟市面教程最不一样的地方。

打开任何一篇「教你做 AI 知识库」的文章，第一句话都是「先装个向量库」「上 embedding」「学 RAG」——听着特别高大上，但普通人根本搞不定：向量库要部署要维护、embedding 模型要选要跑、RAG 链路要调要测、出问题没人帮你查。

flux 一个都没用。反而效果更好。

反共识一：「标签」是 AI 帮你打的，已经做过一次语义提炼

大家以为标签是死的——「我手打 #AI、#知识库 这种关键词，能找到什么？」

但 flux 的标签不是我自己手打的，是 Claude 在 ingest（把一条新内容收进库、加工入库的那道工序）每篇笔记时自动打的：它读完原文、理解了核心观点，然后从一套预定义的标签体系里挑出 3-5 个最贴切的。

这意味着——**标签本身就携带了语义判断**。两条笔记如果共享 3 个标签，几乎可以肯定它们在讲相关的事。这跟「我自己手打关键词然后 grep」完全不是一码事。

反共识二：「土办法」打分推荐，比向量更可解释

flux 找关联用的是一个 AI 写的 Python 小脚本——逻辑简单到尴尬：

- 1 拿一条新笔记
- 2 跟 `notes/` 下所有老笔记**两两比较**
- 3 算两个指标：标签重叠数 + 标题关键词重叠数
- 4 加权打分，分数 ≥ 4 的推荐为关联
- 5 双向写入 frontmatter 的 `related` 字段

就这么土。但用下来——比我之前试过的向量召回都准。

为啥？

- 标签已经是语义压缩过的（上一节讲了，AI 帮你打）
- 标题是你自己最浓缩的判断，标题词重叠通常指向同一个底层主题
- 可解释——分数怎么算的清清楚楚，不像向量库甩你一个「相似度 0.83」的鬼话

向量库不是不能用——只是个人知识库这个级别，相当于用大炮打蚊子。

反共识三：把关联双向写入文件，AI 查询时直接读

很多 RAG 系统是这样：用户问问题 → 实时跑向量召回 → 拿 top-k 文档 → 喂给 LLM。

flux 倒过来——关联在 ingest 时就预先算好、写进 frontmatter：

```
yaml

---
title: Karpathy LLM Wiki 理念
related:
  - 2026-05-10-cyrilXBT-JARVIS知识系统.md
  - 2026-05-08-Garry-Tan-GBrain操作系统.md
---
```

而且双向——A 关联 B 的同时，B 也要关联 A。

这样的好处：

- AI 读一篇笔记时直接看得到关联，跳着读就行，不需要再调一次召回
- 关联是结构化数据，能用 `git diff` 看「这次 ingest 改了哪些关联」
- 每条关联背后都有理由（来自那个打分脚本的日志）

一句话讲完这套机制

AI 帮你打标签 → 脚本算重叠 → 双向写入文件 → 关联就出来了，全程不用向量库。



如果你做的是个人知识库、个人项目，**强烈建议先试这套轻量办法**——它够用、好理解、零运维。等你的库长到 10 万篇了，再考虑换向量库（实话说，你大概率永远到不了那一天）。

💡 **一句话总结：**找关联不一定要上向量——*AI 打的标签 + 文件里写好的 `related` 字段，已经能跑出一个比 RAG 更准、更可解释的关联系统。*

08 怎么让 AI 帮你选型

原则讲完了，落到实操就一件事：**怎么让 AI 推荐适合你的栈。**

用 08 的架构文档当底料

08 让你产出了一份架构文档（进 / 存 / 取 + 数据流图）。这份文档就是给 AI 选型的**最好底料**——它知道你要做什么、东西怎么走，自然能推荐合适的工具。

直接把架构文档丢给 AI，让它出选型建议。提示词参考：

纯文本

我现在有一份产品架构文档（贴下面），请帮我推荐一套技术栈。

约束：

1. 我不是程序员，需要选 AI 最熟、社区最大的工具
2. 优先本地能跑、零运维的方案
3. 数据存储优先文件，能不用数据库就不用
4. 给我 2-3 个候选方案，标清楚每个方案的优劣

【架构文档贴在这里】

怎么判断 AI 推荐的靠不靠谱

AI 会给你一堆名字，你不熟也不知道哪个好。两个动作：

- 1 **去 GitHub 搜一下这个工具的名字**，按上面三个信号（star / 更新 / issue）过一遍
- 2 **追问 AI：**「你推荐的 X 和 Y 比，哪个更适合非程序员？哪个 AI 写代码错误率更低？」

让 AI 自己给自己投票——它会告诉你哪个更主流。

一个常见的坑

AI 有时候会推荐**最新最潮**的工具（因为社区在热议），但其实文档少、生态不成熟。

我的经验是：**追问一句「这个工具发布多久了？有没有更成熟的替代品？」**——通常 AI 会立刻改口推荐一个更老更稳的方案。

💡 **一句话总结：**让 AI 推选型，但要追问、要交叉验证、要去 GitHub 自己看一眼——这是你作为采购员的尽职调查。

09 动手：让 AI 出一份「架构 + 选型」初稿

到这一步，你应该已经有：

- 07 产出的一页纸 MVP 定义
- 08 产出的架构文档（数据流图 + 进存取分析）

现在加最后一步——让 AI 在架构文档基础上补一份**选型清单**。

直接打开 Claude / ChatGPT，贴下面的提示词：

纯文本

你是我的产品技术顾问。我是非程序员，正在用 AI 做我的第一个产品。

下面是我的产品 MVP 定义和架构文档：
【贴 07 + 08 的产出】

请帮我：

1. 在这份架构每一层（进 / 存 / 取）补上推荐的技术栈

2. 每个推荐说清楚：为啥选它、有没有更简单的替代

3. 整体遵守四个原则：AI 最熟 / 本地能跑 / 文件优先 / 社区大

4. 给我一份「最简版本」的清单--能不用就不用、能少就少

5. 最后列一份「我现在还需要决定什么」的开放问题清单

AI 出完之后，你做三件事：

- 1 把每个推荐的工具名字去 GitHub 验一遍（三个信号过一遍）
- 2 你看不懂的工具，追问 AI「这个能用更简单的替代吗？」
- 3 最终敲定一份选型清单，存到项目里（我建议命名 TECH-STACK.md，跟 ARCHITECTURE.md 放一起）

到这，你就有了完整的开干前三件套：

- MVP 定义（07）：做什么
- 架构文档（08）：东西怎么走
- 选型清单（09）：用什么工具走

下一篇 10，我们就拿着这三件套，正式开 Vibe Coding。

💡 一句话总结：动手不是自己研究技术，是让 AI 在架构文档上补选型 → 你拿三个信号验 → 存成选型清单。

10 小结

这一篇核心就一句话：

AI 时代选技术，不是去学技术，是去当采购员——挑现成的、AI 熟的、社区大的工具，让 AI 帮你组装。

四原则记牢：

原则	怎么做
主流	选 AI 训练数据多的语言（Python / TypeScript）
本地	能不上云就别上
简单	文件 > 数据库，能不用就不用
大众	star 多 + 更新近 + 维护者活

到这里，你手上应该已经有了：

- 06：为什么做（动机）

- 07: 做什么 (一页纸 MVP)
- 08: 怎么走 (架构 v1)
- 09: 用什么搭 (选型清单) ← 你在这

下一篇 10 「Vibe Coding 实战: 从架构到第一版能跑起来的代码」——三件套齐了, 该正式开干了。我会带你看一次完整的 vibe coding 过程: 怎么给 AI 喂上下文、怎么分步推进、怎么验证每一步、怎么调 bug, 以 fuxi 的某个模块为案例, 从 0 写到能跑。

本系列持续更新, 建议收藏。

